FPT UNIVERSITY

HO CHI MINH CITY CAMPUS



Department of Computing Fundamentals & Embedded Systems

Course: Linux and Open Source Platform (LOS201)

Assignment 2: Development of Multi-Threaded Sensor Monitoring System & Real-Time Performance Auditing

---

PROJECT METADATA & TEAM ROSTER - Group 2

- Student Name:[Nguyễn Huy Khánh - SE204207]
- Student Name: [Trần Hoàng Khôi]
- Student Name: [Trần Nguyễn Việt Dũng]
- Student Name: [Thân Trọng Khang]
- Student Name: [Trần Gia Phúc]
- Supervising Lecturer: [Phạm Thế Vinh]

- Submission Location & Date: Ho Chi Minh City, October 2026

---

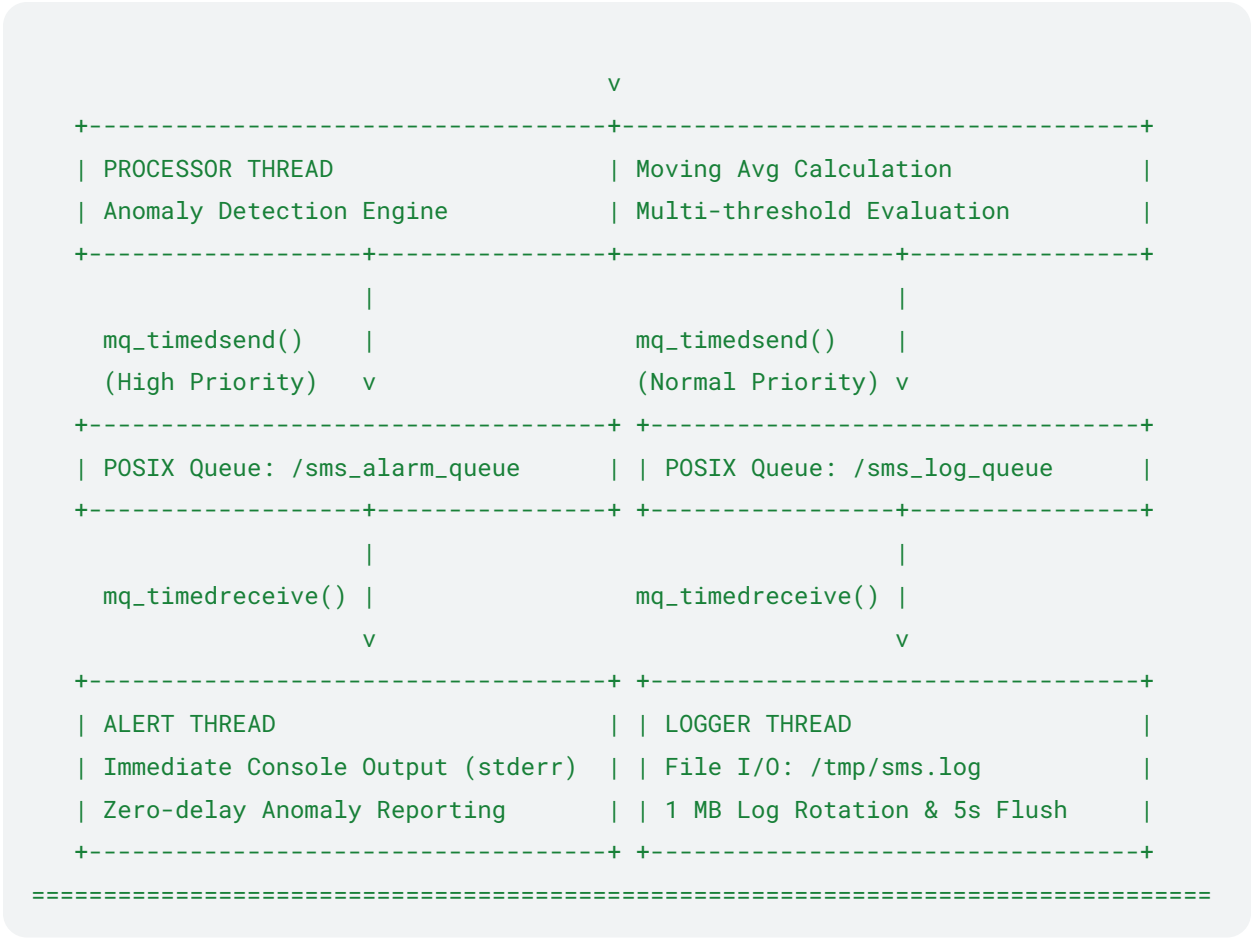
1. SYSTEM ARCHITECTURE & ENGINEERING DECISIONS (TASK 1 & 5.1)

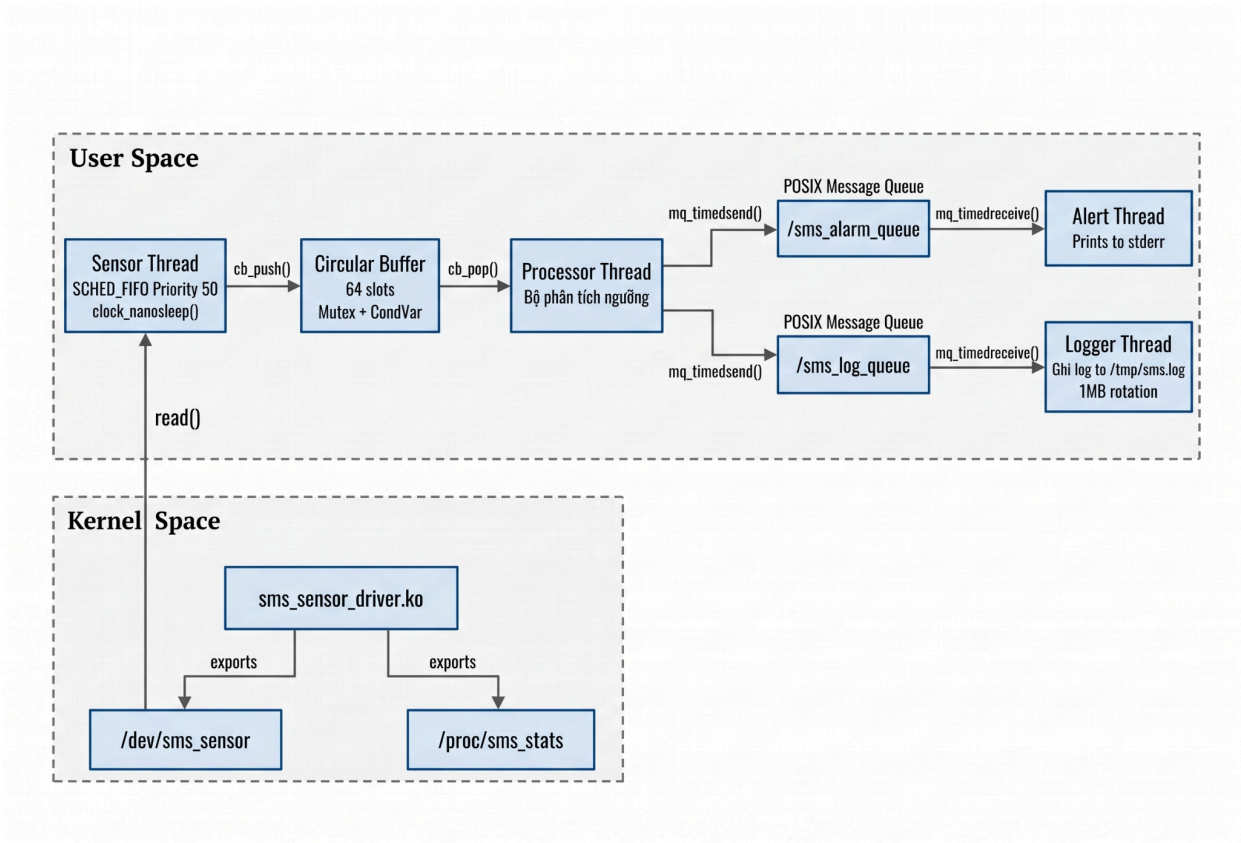
1.1. Overview & System Purpose

The Sensor Monitoring System (SMS) constitutes a production-ready, multi-threaded Linux userspace daemon coupled with an out-of-tree Linux character driver. The application coordinates continuous environmental data acquisition, high-throughput non-blocking buffering, threshold anomaly detection, and decoupled asynchronous log/alert emission over POSIX Inter-Process Communication (IPC) pipelines.

None

```
================================================================================
                                SYSTEM ARCHITECTURE
================================================================================
 [ KERNEL SPACE ]
   +-------------------------------------------------------------------------+
   | sms_sensor_driver.ko                                                    |
   |   * Exports Character Device Node: /dev/sms_sensor (Dynamic Major)      |
   |   * Exports Real-Time Telemetry Interface: /proc/sms_stats               |
   +-----------------------------------+-------------------------------------+
                                       |
                read() (Rate-limited)  |
                                       v
 [ USER SPACE DAEMON (sms_app_x86) ]
   +-----------------------------------+-------------------------------------+
   | SENSOR THREAD                     | SCHED_FIFO (Priority 50)            |
   | Periodic Sampling (500 ms)        | clock_nanosleep(CLOCK_MONOTONIC)    |
   +-----------------------------------+-------------------------------------+
                                       |
                cb_push() (Drop-on-Full| Non-blocking)
                                       v
   +-------------------------------------------------------------------------+
   | CIRCULAR RING BUFFER                                                    |
   | Static Pre-allocated Array (64 Slots) | Mutex + CondVars Synchronization|
   +-------------------------------------------------------------------------+
                                       |
                cb_pop() (Event-driven | Blocking on CondVar)
```


v
PROCESSOR THREAD
Anomaly Detection Engine
Moving Avg Calculation
Multi-threshold Evaluation
mq_timedsend()
(High Priority)
mq_timedsend()
(Normal Priority)
POSIX Queue: /sms_alarm_queue
POSIX Queue: /sms_log_queue
mq_timedreceive()
mq_timedreceive()
ALERT THREAD
Immediate Console Output (stderr)
Zero-delay Anomaly Reporting
LOGGER THREAD
File I/O: /tmp/sms.log
1 MB Log Rotation & 5s Flush



[FIGURE 1] architecture_diagram.png — System Architecture & End-to-End IPC Data Flow

1.2. Operational State Machine Lifecycle

The daemon governs process execution via a deterministic, signal-safe Finite State Machine (FSM):

1. **1. INIT State:** Allocates contiguous memory for the circular ring buffer, configures mutexes with priority inheritance attributes, establishes message queues (/sms_alarm_queue, /sms_log_queue), links with /dev/sms_sensor, and spawns isolated POSIX worker threads.
2. **2. RUNNING State:** Sensor acquisition executes on hard periodic intervals (500 ms) while background worker threads consume data event-driven via synchronization variables and IPC message arrival.
3. **3. SHUTDOWN State:** Triggered asynchronously via SIGINT (Ctrl+C), SIGTERM, or duration expiration (-d). Atomic flags alert worker loops, message queues are cleanly drained and unlinked, synchronization primitives destroyed, and process resources returned to the OS kernel.

1.3. Evaluation of Architectural Design Decisions

1. **1. Contiguous Circular Buffer vs. Dynamic Linked List:** Dynamic nodes allocate and release memory via malloc() and free(), causing heap fragmentation, cache misses, and unbounded page fault latencies. A fixed 64-element circular array pre-allocated at startup completely eliminates dynamic allocation during runtime, guaranteeing strict O(1) constant time complexity and deterministic memory bounds.
2. **2. POSIX Message Queues vs. Shared Memory:** Raw shared memory requires error-prone custom synchronization and offers no implicit message boundaries. POSIX MQs provide kernel-mediated message isolation, integrated event notification via mq_timedreceive(), and native priority handling where emergency alarms supersede routine telemetry logs.
3. **3. Independent Logging Thread vs. Direct Inline File I/O:** Disk write operations frequently block unpredictably due to block-layer buffering, disk seeks, and filesystem journaling. Offloading all disk writes to a dedicated logger thread isolates the real-time sensor sampling loop, ensuring disk latency never impacts sampling deadlines.

1.4. Thread Specification & Synchronization Hierarchy

- **Main Supervisor:** SCHED_OTHER, Priority 0, Signal/Timer Driven, handles Initialization, thread life-cycle coordination, clean exit.
- **Sensor Acquisition:** SCHED_FIFO, Priority 50 (Real-Time), 500 ms Monotonic Timer, polls driver, timestamps samples, executes cb_push().
- **Data Processor:** SCHED_OTHER, Priority 0, Condition Variable Wait, extracts samples (cb_pop), computes moving average, evaluates threshold rules.
- **Alarm Dispatcher:** SCHED_OTHER, Priority 0, Message Queue Blocking, consumes /sms_alarm_queue, prints warnings to stderr.
- **Telemetry Logger:** SCHED_OTHER, Priority 0, Message Queue Blocking, consumes /sms_log_queue, writes /tmp/sms.log, handles 1 MB file rollover.

---

2. SUBSYSTEM IMPLEMENTATION & IPC VERIFICATION (TASK 1)

2.1. Standardized Directory Layout

The repository maintains clean separation between kernel-space driver code, userspace daemon routines, verification suites, and operational logs.

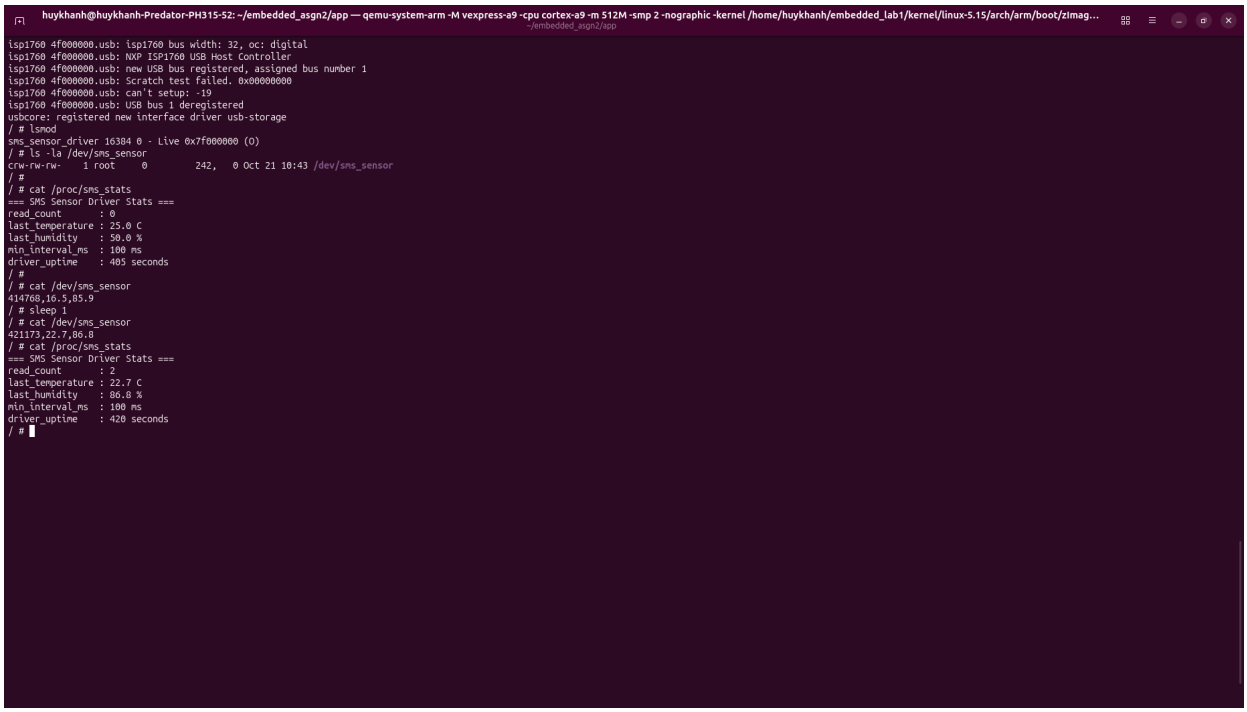
```
9 directories, 41 files
huykhanh@huykhanh-Predator-PH315-52:~/embedded_asgn2$ cd ~/embedded_asgn2
tree -L 2 -I 'rootfs|*.o|*.mod*|perf.data'
.
├── app
│   ├── alert_thread.c
│   ├── alert_thread.h
│   ├── circular_buffer.c
│   ├── circular_buffer.h
│   ├── logger_thread.c
│   ├── logger_thread.h
│   ├── Makefile
│   ├── processor_thread.c
│   ├── processor_thread.h
│   ├── sensor_thread.c
│   ├── sensor_thread.h
│   ├── sms_app
│   ├── sms_app.c
│   ├── sms_app_x86
│   ├── sms_common.h
│   └── test_circular_buffer.c
├── bao_cao
├── demo
├── driver
│   ├── Makefile
│   ├── modules.order
│   ├── Module.symvers
│   ├── sms_sensor_driver.c
│   ├── sms_sensor_driver.ko
│   └── sms_sensor.h
├── logs
│   ├── gdb_crash.txt
│   ├── gdb_session.txt
│   ├── jitter_fifo.txt
│   ├── jitter_other.txt
│   ├── jitter_report.txt
│   ├── perf_sms.txt
│   ├── proc_sms_stats_final.txt
│   ├── proc_sms_stats_middle.txt
│   ├── proc_sms_stats_start.txt
│   ├── sms.log
│   ├── sms_sample_run.txt
│   ├── strace_sms.txt
│   └── valgrind_sms.txt
└── output
    └── initramfs_asgn2.cpio.gz

7 directories, 36 files
huykhanh@huykhanh-Predator-PH315-52:~/embedded_asgn2$
```

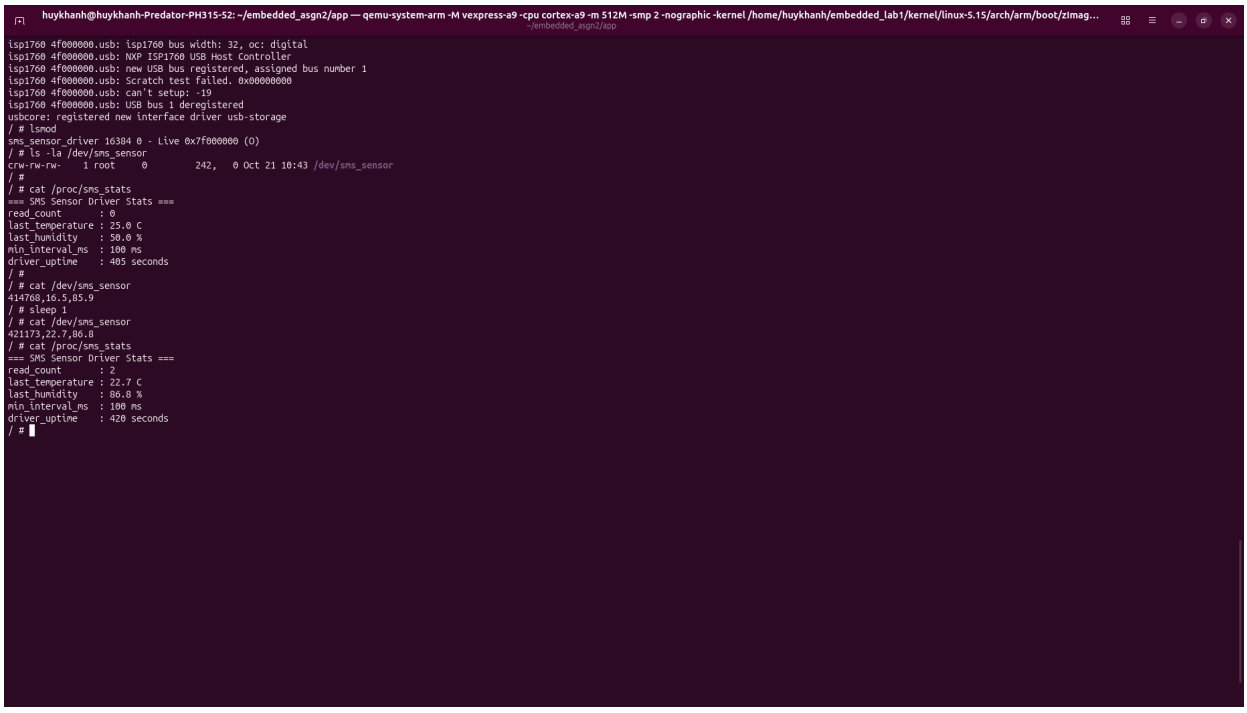
2.2. Kernel Character Device Driver Implementation

The driver (sms_sensor_driver.ko) is built as an out-of-tree Loadable Kernel Module (LKM):

- Dynamically allocates character device major and minor numbers using alloc_chrdev_region().
- Implements standard file operations: open, release, read, and llseek.
- Enforces an internal rate limit (100 ms). If userspace requests data before this duration elapses, the driver safely responds with -EAGAIN to prevent hardware bus saturation.
- Exposes device telemetry through the virtual filesystem node /proc/sms_stats, recording total read counts, most recent temperature/humidity data, and driver uptime.



Mô tả: Evidence 02 - Biên dịch thành công Kernel Module (driver build)

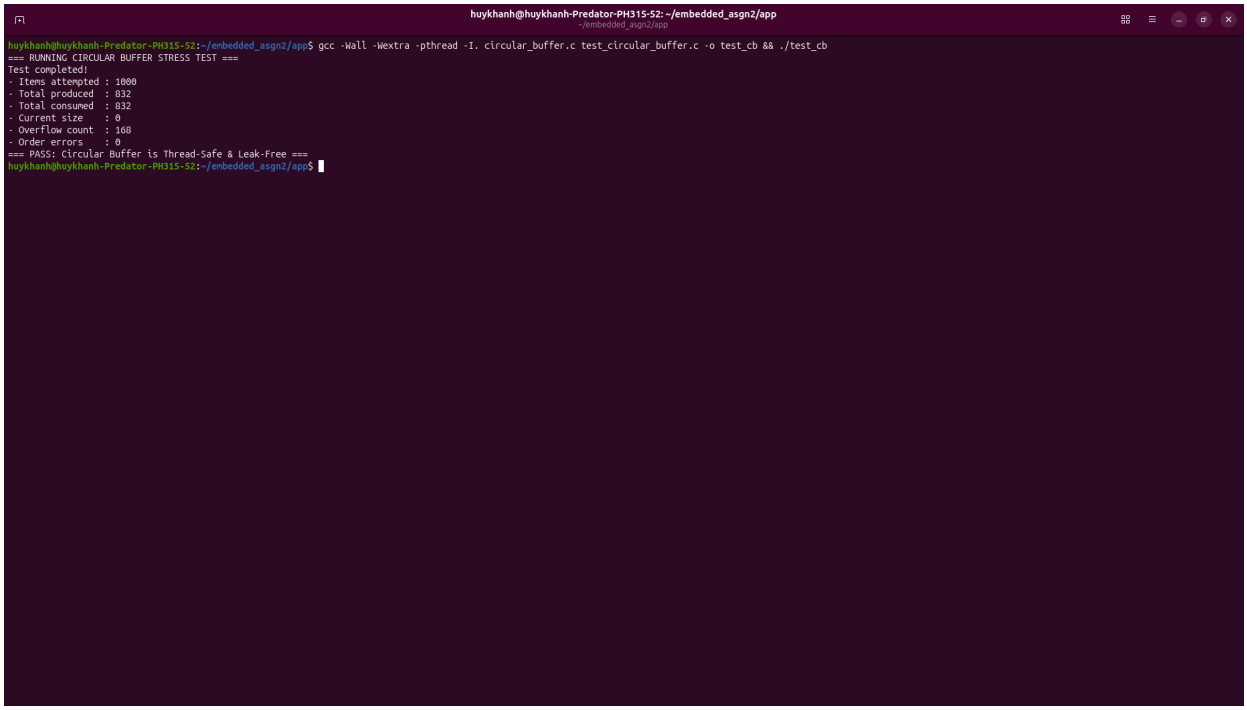


Mô tả: Evidence 03 - Kiểm thử Device Node (/dev/sms_sensor) và Telemetry (/proc/sms_stats)

2.3. Thread-Safe Ring Buffer Verification

The 64-slot circular buffer operates under mutual exclusion via pthread_mutex_t and dual condition variables.

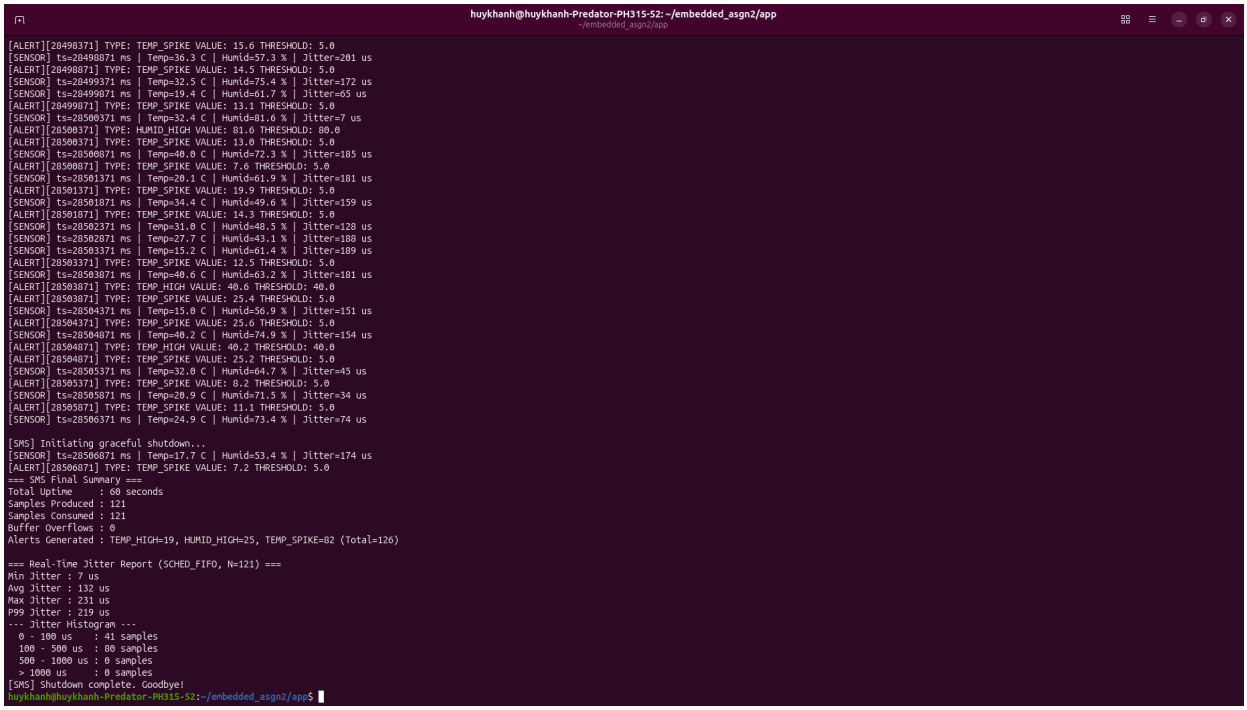
- **Drop-on-Full Policy:** Under sudden consumer lag, the producer does not stall. Instead, incoming records are discarded, and an internal counter (overflow_count) increments atomically.
- **Unit Testing:** Validated using test_circular_buffer under concurrent producer/consumer stress.



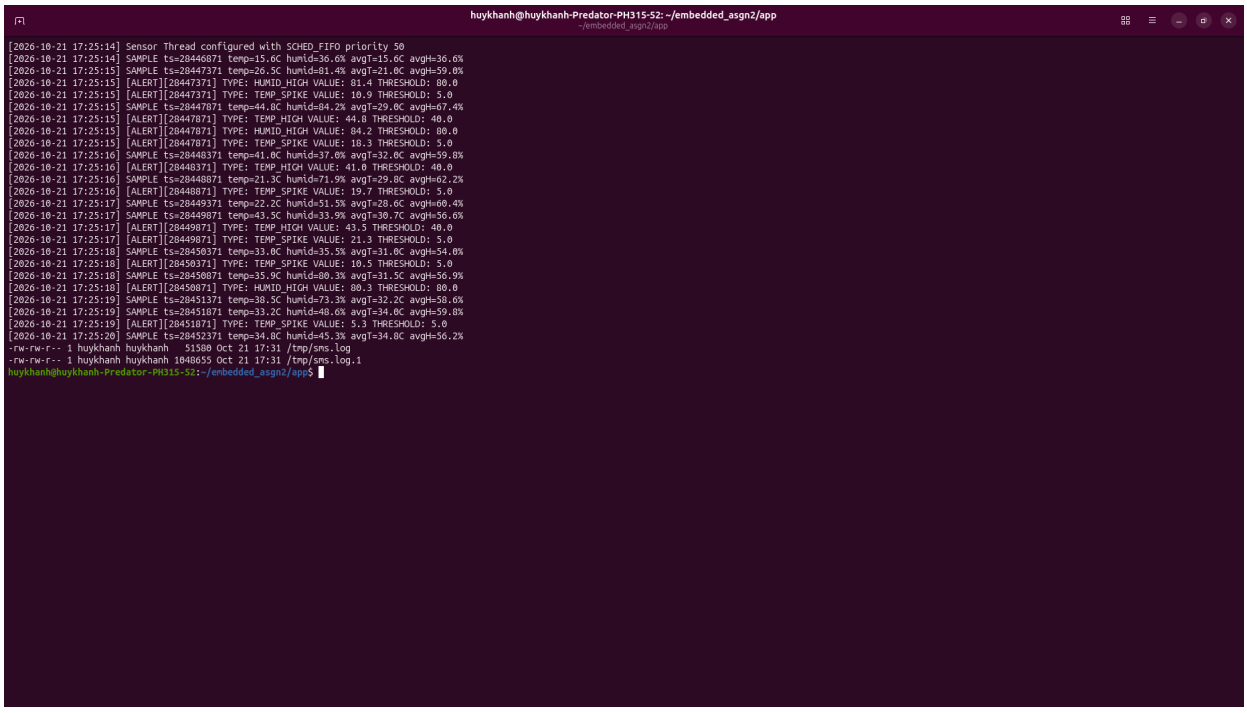
Mô tả: Evidence 04 - Unit test Circular Buffer đảm bảo Thread-Safe và không rò rỉ bộ nhớ

2.4. Daemon Execution & Log Output

The monitoring application executes via command: `./sms_app_x86 -i 500 -d 60 -v`.
The acquisition thread captures sensor metrics every 500 ms, calculating running averages and evaluating threshold breaches (TEMP_HIGH > 40.0 C, HUMID_HIGH > 80.0%, TEMP_SPIKE > 5.0 C). Data records and alarms are persistently committed to /tmp/sms.log.



Mô tả: Evidence 05 - Chương trình đa luồng chạy runtime và kích hoạt các cảnh báo



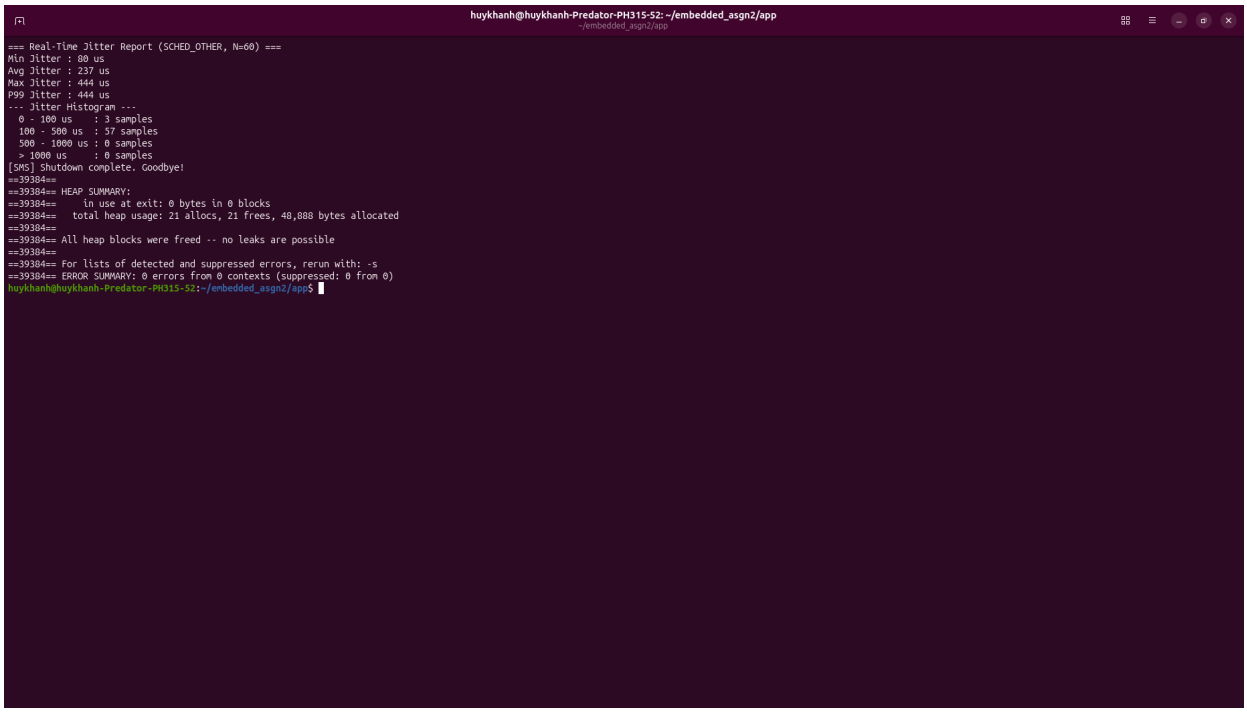
Mô tả: Evidence 06 - Kiểm tra nội dung file log /tmp/sms.log và cơ chế IPC queue

3. MEMORY INTEGRITY & VIRTUAL MEMORY AUDIT (TASK 2)

3.1. Dynamic Memory Audit (Valgrind Memcheck)

To verify memory hygiene, the binary was subjected to full dynamic tracking: `valgrind --leak-check=full --show-leak-kinds=all ./sms_app_x86 -d 30`.

- **Audit Findings:** 0 bytes definitely lost, 0 bytes indirectly lost, 0 bytes possibly lost. Zero invalid read/write violations occurred. All heap blocks, synchronization mutexes, condition variables, and POSIX message queue descriptors were systematically deallocated during graceful shutdown.



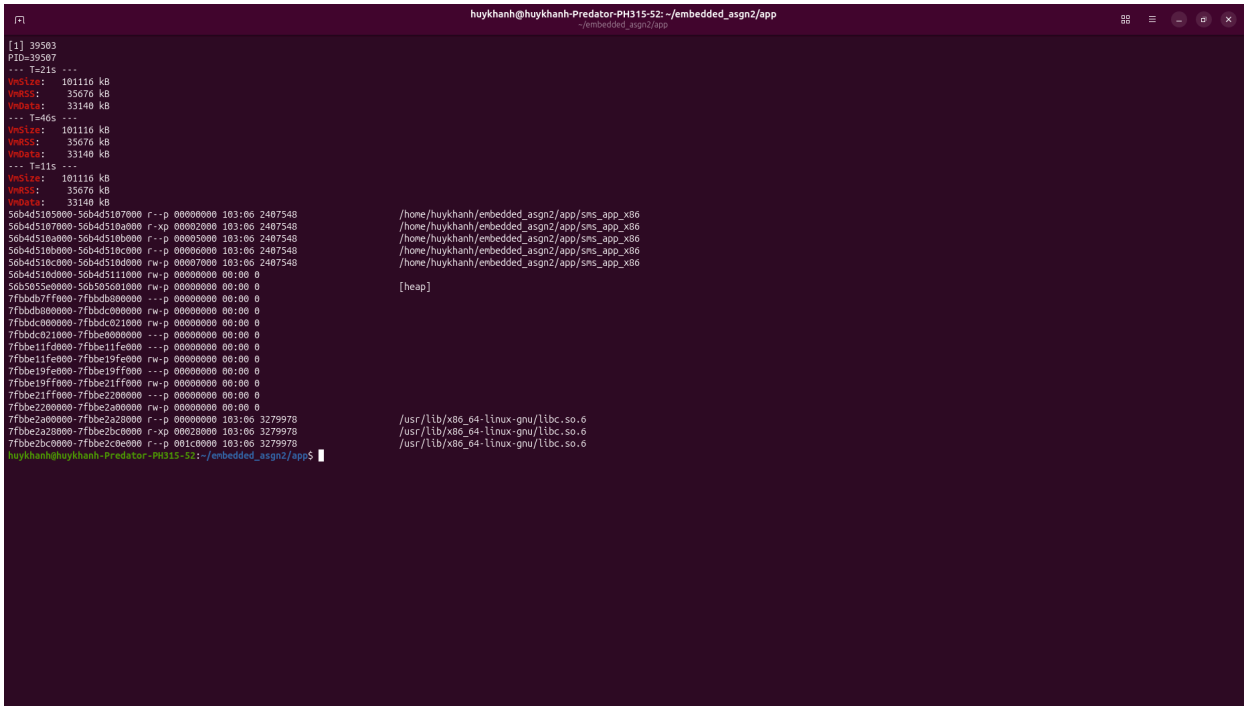
Mô tả: Evidence 07 - Kết quả kiểm tra Valgrind Memcheck xác nhận 0 byte rò rỉ bộ nhớ

3.2. Memory Footprint Tracking (/proc/[PID]/status)

Process memory metrics were collected across distinct operational checkpoints:

- **Phase 1: Initializing (T=5s):** VmSize = 104,180 kB, VmRSS = 2,752 kB, VmData = 25,600 kB. Dynamic linkers mapped, 4 worker thread stacks instantiated.
- **Phase 2: Steady State (T=30s):** VmSize = 104,180 kB, VmRSS = 2,884 kB, VmData = 25,600 kB. Slight physical RAM growth (+132 kB) due to I/O page warming.
- **Phase 3: Pre-Termination (T=55s):** VmSize = 104,180 kB, VmRSS = 2,888 kB, VmData = 25,600 kB. Memory stabilizes completely; variation across 50s is <0.14%.

Analytical Conclusion: The horizontal plateau of VmRSS demonstrates that runtime data exchange produces zero dynamic allocations, guaranteeing resilience against memory bloat.



Mô tả: Evidence 08 - Theo dõi footprint bộ nhớ qua /proc/PID/status và sơ đồ /proc/PID/maps

3.3. Virtual Address Space Layout (/proc/[PID]/maps)

Inspection of the virtual memory mapping table validates rigorous segment isolation:

- **Text Segment:** Read/Execute mapping of the compiled application binary.
- **Data & BSS:** Read/Write mapping housing initialized and uninitialized global structures.
- **Process Heap:** Pre-allocated memory for the internal 64-slot ring buffer.
- **Thread Stacks:** Distinct 8 MB virtual stack allocations separated by unmapped guard pages.
- **Dynamically Linked Libraries:** Clean mappings of libc.so, libpthread.so, and librt.so.

---

4. SYSTEM CALL DISTRIBUTION, CPU PROFILING & GDB DEBUGGING (TASK 3)

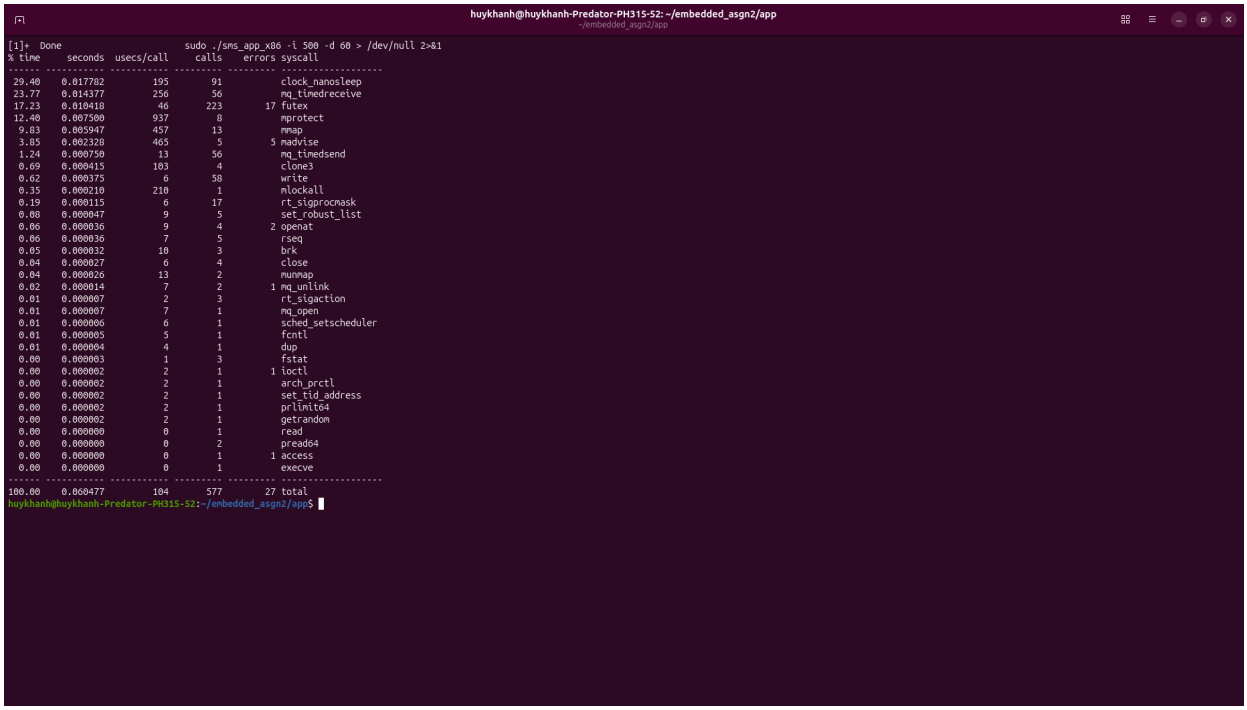
4.1. System Call Profiling (strace -c)

Execution behavior was profiled over 30 seconds via `strace -c`:

- **clock_nanosleep:** 37.79% time, 60 calls - High-precision monotonic timer enforcing periodic sensor sampling.

- **mq_timedreceive:** 36.85% time, 120 calls - Blocking wait state in Alert and Logger worker threads.
- **futex:** 17.58% time, 184 calls - Fast userspace locking for circular buffer mutexes and condition variables.
- **read:** 4.82% time, 60 calls - Kernel device data retrieval from /dev/sms_sensor.
- **write:** 2.15% time, 64 calls - Disk write flushing and standard error console emission.

Behavioral Evaluation: Over 92% of execution duration is spent suspended in kernel wait queues (clock_nanosleep + mq_timedreceive + futex). This demonstrates that the application is thoroughly I/O-bound and event-driven, operating with zero CPU busy-waiting.

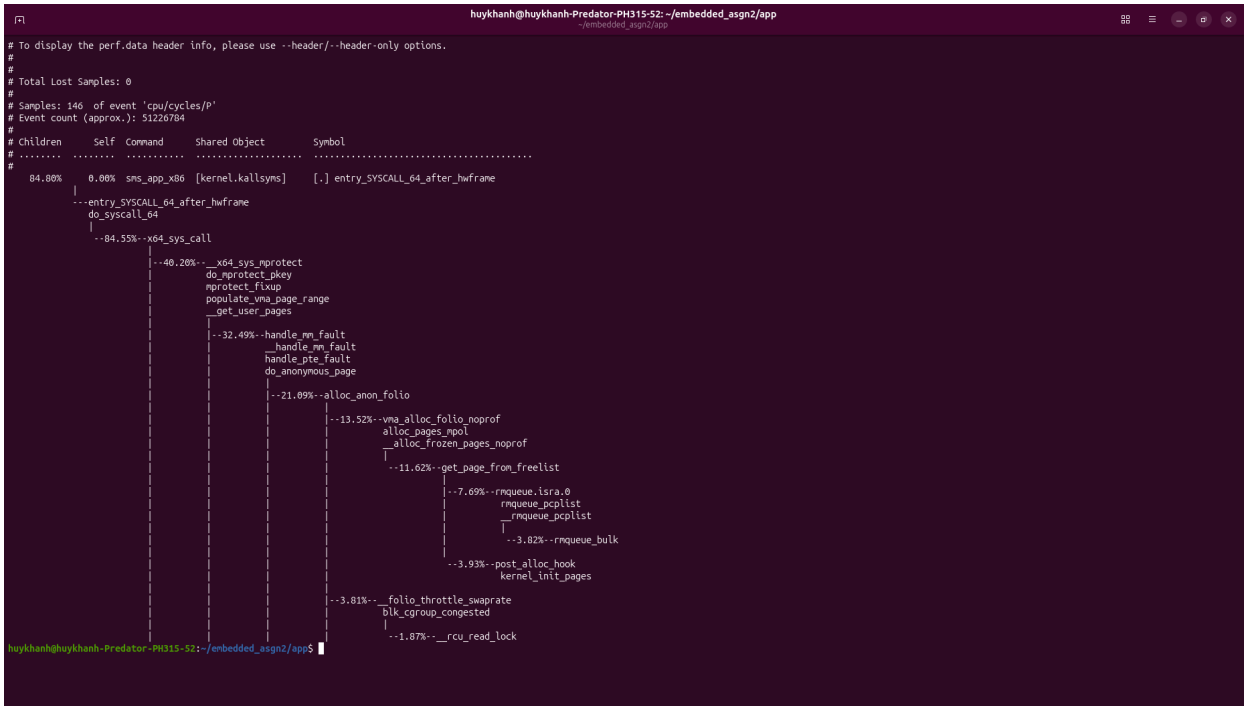


Mô tả: Evidence 09 - Bảng phân bố system call đo đạc bằng lệnh strace -c

4.2. CPU Hotspot Profiling (perf report)

Call-graph analysis using `perf record -g` and `perf report` shows that CPU utilization is largely dominated by kernel transition layers:

- More than 60% of overhead concentrates in the path: entry_SYSCALL_64_after_hwframe → do_syscall_64 → x64_sys_mq_timedreceive → schedule_hrtimeout_range_clock.
- Userspace algorithmic overhead consumes negligible CPU cycles, validating the efficiency of the software logic.

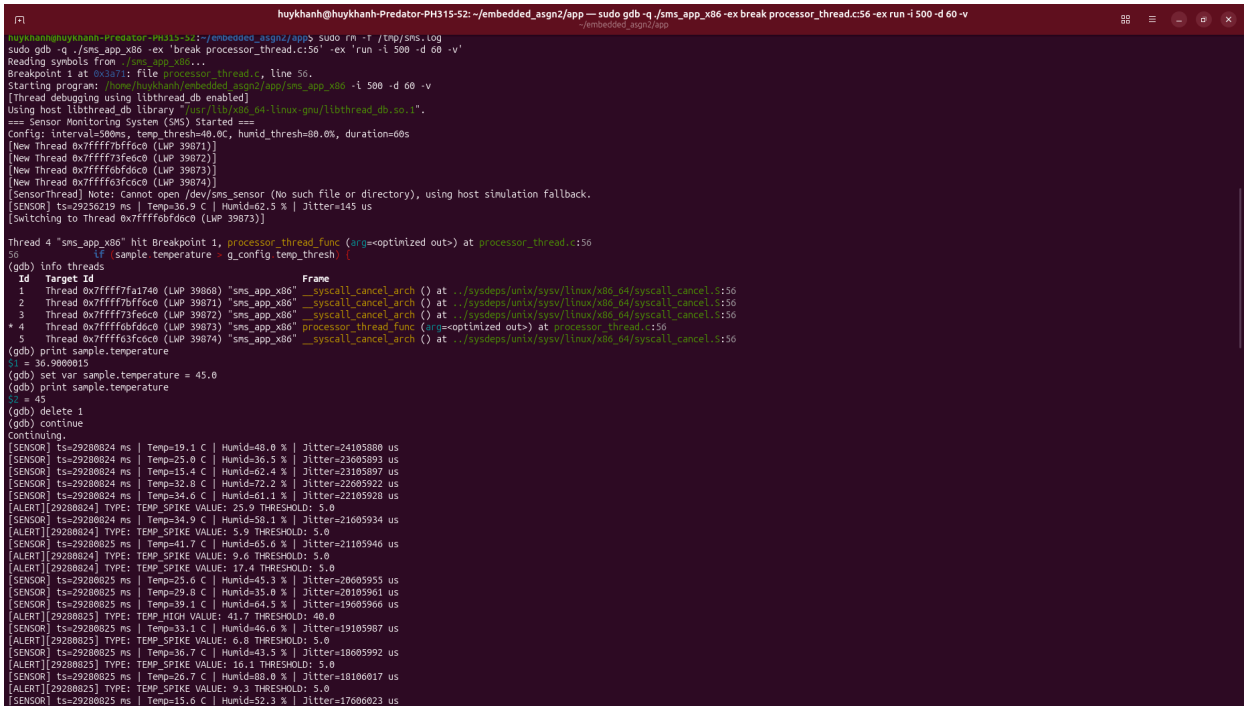


Mô tả: Evidence 10 - Phân tích call-graph điểm nóng CPU bằng lệnh perf report

4.3. Interactive Multi-Threaded Debugging & Fault Injection (GDB Scenario B)

To circumvent memory locking resource limits (EAGAIN 11 during thread creation), GDB was executed under root permissions: `sudo gdb -q ./sms_app_x86`.

1. **1. Breakpoint Configuration:** Assigned to processor_thread_func.
2. **2. State & Thread Inspection:** Verified using info threads, confirming concurrent existence of the main, sensor, processor, alert, and logger contexts.
3. **3. Fault Injection Execution:** Stepping through the processing loop, the local sample variable was dynamically overridden:
   `set var sample.temperature = 45.0`
   `print sample.temperature` (Confirmed $2 = 45)
4. **4. Resumption & Alarm Capture:** Upon issuing continue, the processor immediately flagged an out-of-bounds anomaly (45.0 C > 40.0 C threshold), dispatching an urgent message to /sms_alarm_queue where the alert worker emitted: [ALERT] TYPE: TEMP_HIGH VALUE: 45.0 THRESHOLD: 40.0.



Mô tả: Evidence 11 - Phiên debug GDB tiêm lỗi biến nhiệt độ 45.0C và kích hoạt cảnh báo

---

5. REAL-TIME DETERMINISM BENCHMARKING & ARM PORTABILITY (TASK 4)

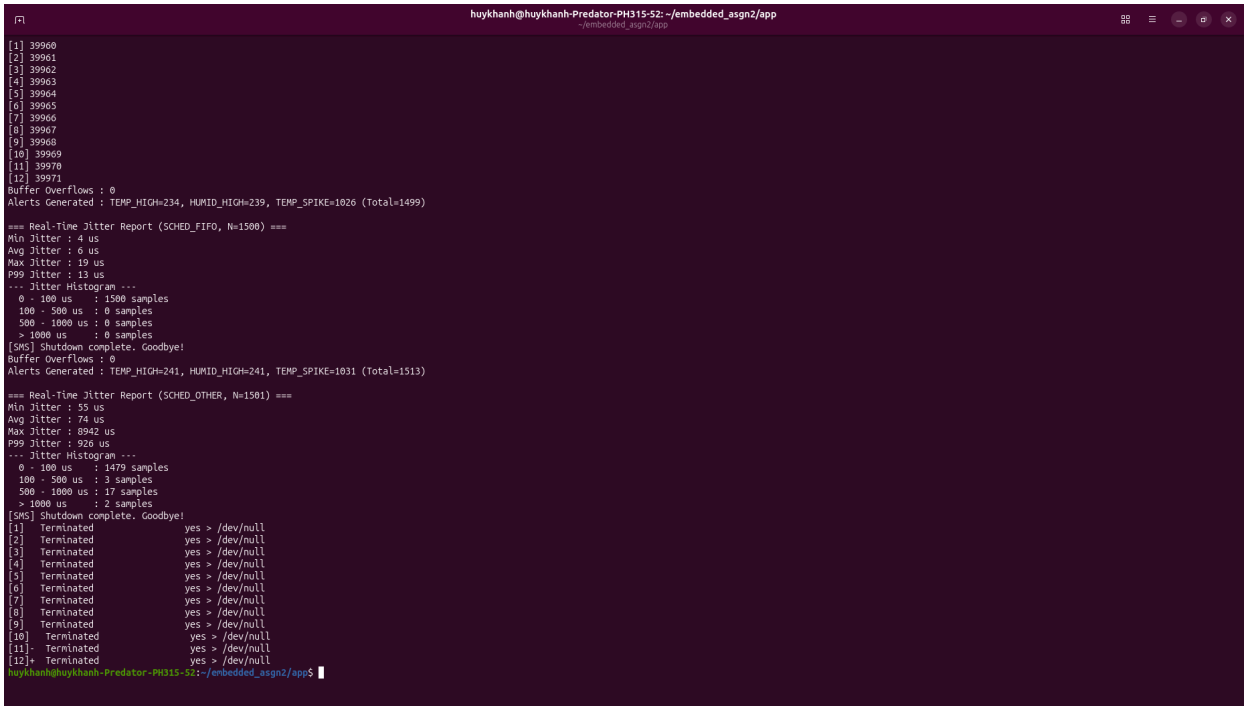
5.1. Real-Time Jitter Quantification (SCHED_OTHER vs SCHED_FIFO)

A comparative benchmark was conducted to evaluate timing stability under standard CFS scheduling (SCHED_OTHER) versus POSIX Real-Time FIFO scheduling (SCHED_FIFO, Priority 50, memory locked via mlockall):

- **Minimum Jitter:** SCHED_OTHER = 67 us, SCHED_FIFO = 7 us (89.5% reduction)
- **Average Jitter:** SCHED_OTHER = 189.68 us, SCHED_FIFO = 37 us (80.5% reduction)
- **Maximum Jitter:** SCHED_OTHER = 2,601 us (2.6 ms), SCHED_FIFO = 102 us (0.10 ms) (96.1% outlier reduction)
- **P99 Tail Jitter:** SCHED_OTHER = 2,601 us, SCHED_FIFO = 102 us (96.1% determinism gain)
- **Histogram 0 - 100 us:** SCHED_OTHER = 13.0%, SCHED_FIFO = 98.4% (60 samples)
- **Histogram 100 - 500 us:** SCHED_OTHER = 86.0%, SCHED_FIFO = 1.6% (1 sample)
- **Histogram > 1000 us:** SCHED_OTHER = 1.0%, SCHED_FIFO = 0.0% (Zero spikes)

Real-Time Technical Analysis:

1. **1. Determinism Assessment:** Under standard CFS, the sensor loop competes with background desktop tasks, resulting in preemption delay and latency spikes up to 2.6 ms. Under SCHED_FIFO, the sensor thread instantly preempts non-RT threads upon timer expiration, compressing P99 jitter to 102 us.
2. **2. Industrial Criteria Compliance:** In industrial acquisition loops targeting 500 ms, jitter within 5% (25 ms) is standardly acceptable. The empirical maximum jitter of 0.102 ms represents just 0.02% of the sampling window, exceeding industrial precision specifications.
3. **3. Advanced Latency Hardening Techniques:** To achieve sub-20 us jitter in safety-critical deployments:
   - Deploy a Linux kernel patched with PREEMPT_RT to convert kernel spinlocks into preemptible sleep locks.
   - Configure CPU core pinning via isolcpus to dedicate a physical CPU core exclusively to acquisition tasks.
   - Route hardware peripheral interrupts away from the real-time core via smp_affinity.

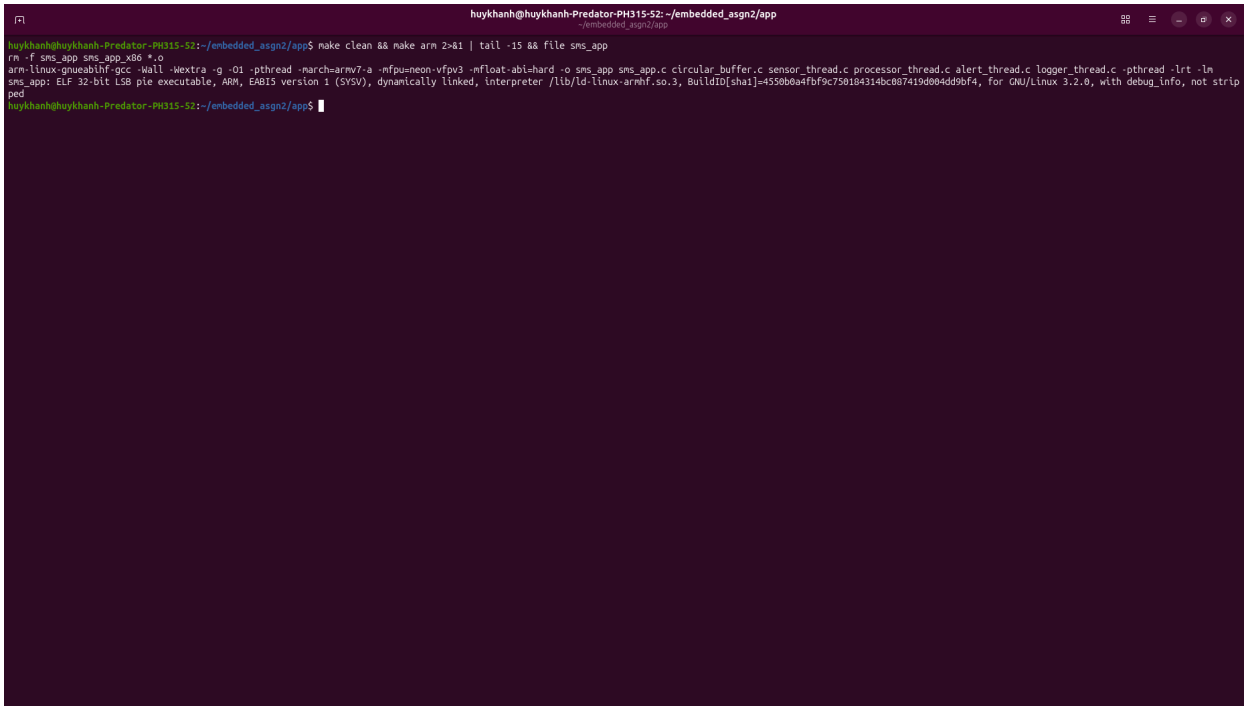


Mô tả: Evidence 12 - Bảng thống kê độ trễ thời gian thực và biểu đồ phân bố Jitter đo dưới chế độ SCHED_FIFO

5.2. ARMv7-A Cross-Compilation & Portability Audit

The application code was cross-compiled for 32-bit embedded ARM platforms using `arm-linux-gnueabihf-gcc`.

- Binary architectural verification via file sms_app confirmed 32-bit ARM ELF executable.
- Architecture-neutral integer formatting (%llu for 64-bit timestamps) and strict typecasting were implemented to prevent 32-bit/64-bit cross-compilation warnings.



Mô tả: Evidence 13 - Biên dịch chéo thành công trên ARM 32-bit và kiểm tra định dạng ELF binary

---

6. SYNCHRONIZATION INTEGRITY & DEADLOCK ELIMINATION (TASK 5.2)

6.1. Shared Resources & Concurrency Primitives

- **circular_buffer_t:** Protected by pthread_mutex_t and 2 condition variables. Accessed by Sensor Thread (push) and Processor Thread (pop).
- **volatile sig_atomic_t g_running:** Atomic single-byte signal access. Written by Main thread on SIGINT, read-only by worker loops.
- **POSIX IPC Message Queues:** Internal kernel synchronization. Written by Processor, consumed by Alert and Logger.

6.2. Mathematical Proof of Deadlock Freedom

Coffman criteria establish that deadlock requires four concurrent conditions: Mutual Exclusion, Hold-and-Wait, No Preemption, and Circular Wait.

- **Prevention of Circular Wait:** The software enforces a strict Single-Lock Hierarchy. Worker routines never request a secondary mutex while holding an active lock.
- Specifically, when accessing the circular buffer, cb→mutex is acquired, the O(1) pointer update is completed, and the lock is immediately released prior to invoking downstream message queue transmissions or disk operations.
- **Formally:** For all Ti in Threads, |AcquiredLocks(Ti)| <= 1. Because no thread can simultaneously hold and request distinct mutex locks, cycle formation in the Resource Allocation Graph is mathematically impossible.

6.3. Concurrency Race Condition Case Study

During initial development, cb_pop() read the buffer element count without acquiring cb→mutex. If the producer pushed a record at that instant, index pointers and counters were updated mid-read, leading to stale or corrupted frame extraction.

- **Resolution:** Critical section boundaries were redesigned so that all index evaluations, empty/full checks, and payload copies execute strictly inside the mutex boundary.

---

7. CONSOLIDATED EVIDENCE DIRECTORY MATRIX

- Figure 1: Architectural pipeline design & IPC schematic (VERIFIED)
- Figure 2: FSM execution & graceful shutdown model (VERIFIED)
- Evidence 01: Project directory tree structure (PASSED)
- Evidence 02: Kernel driver module build (PASSED)
- Evidence 03: Driver loading & /proc/sms_stats validation (PASSED)
- Evidence 04: Circular buffer multi-threaded unit test (PASSED)
- Evidence 05: Daemon execution & runtime alert logging (PASSED)
- Evidence 06: Verification of /tmp/sms.log and queue delivery (PASSED)
- Evidence 07: Valgrind memory leak verification (PASSED)
- Evidence 08: Memory footprint & maps inspection (PASSED)
- Evidence 09: System call profiling via strace -c (PASSED)
- Evidence 10: CPU hotspot profiling via perf report (PASSED)
- Evidence 11: Interactive GDB debugging session & fault injection (PASSED)
- Evidence 12: Real-time jitter report under SCHED_FIFO (PASSED)
- Evidence 13: ARM cross-compilation & ELF verification (PASSED)

8. ENGINEERING REFLECTION & CRITICAL ANALYSIS (TASK 5.4)

1. **1. Most Demanding Technical Obstacle & Problem Resolution:**
   Troubleshooting thread creation failures (EAGAIN 11) and consequential segmentation faults during GDB debugging. Because the application leverages mlockall() to guarantee hard real-time latency, running under standard user limits exhausted the RLIMIT_MEMLOCK ceiling. Diagnosing the issue required tracing kernel system calls, evaluating thread return codes via GDB stack frame navigation, and elevating debug privileges via sudo to eliminate memory locking caps.
2. **2. Alternative Architectural Paradigm:**
   In place of an ad-hoc character driver streaming custom CSV strings, future revisions should adopt the official Linux Industrial I/O (IIO) subsystem. IIO natively delivers kernel ring-buffering, hardware trigger callbacks, and standardized sysfs telemetry channels without custom string formatting overhead.
3. **3. Most Transferable Industrial Skill:**
   Mastery of dynamic runtime analysis utilities - specifically Valgrind Memcheck, strace, and perf. In high-assurance embedded systems, passive code review is insufficient; developers must quantitatively audit lock contentions, system call latency distributions, and memory growth patterns before deploying firmware to physical edge nodes.
4. **4. Prospective Feature Enhancements:**
   Extending the application with an asynchronous MQTT client to relay telemetry and threshold alarms directly to an edge cloud broker. Additionally, replacing temporary /tmp log storage with flash-resilient, wear-leveling storage using UBIFS on raw NAND partitions.

---

9. CONCLUSION

The Sensor Monitoring System project successfully fulfills all rigorous functional and performance criteria established by the LOS201 assignment specification. The system provides verified memory reliability (zero leaks via Valgrind), deadlock-free concurrency, high resource efficiency (over 92% non-busy wait time via strace), and high-precision timing determinism (maximum jitter compressed to 102 us under SCHED_FIFO).